

Notes (Week 5 Wednesday)

These are the notes I wrote during the lecture.

An implication $\phi \Rightarrow \psi$ is true iff:

whenever ϕ is true, so is ψ

An entailment $T \models \psi$ is valid iff:

in every interpretation $\mathcal{I}$ where
if for each $\phi \in T$. $\llbracket \phi \rrbracket_{\mathcal{I}} = \text{true}$
then $\llbracket \psi \rrbracket_{\mathcal{I}} = \text{true}$

A = I can see you

B = You can see my mirrors

If you can't see my mirrors, then I can't see you

$\neg B \Rightarrow \neg A$

$A \Rightarrow B$

$\neg A \vee B$

Consider the statement "If you can't see my mirrors, then I can't see you". Which of the following are equivalent statements? Select one or more:

- If I can see you, then you can see my mirrors.
- Either you can see my mirrors, or I can't see you.
- If I can't see you, then you can't see my mirrors.
- If you can see my mirrors, then I can see you.
- You can't see my mirrors, and I can't see you.
- Either you can't see my mirrors, or I can't see you.

Wrap up semantics of predicate logic

Natural deduction for predicate logic

A propositional formula is about a
valuation --- a world where there
are boolean variables that may have
different values

A predicate logic formula is about a
interpretation
interpretation = model + environment

Model says:

- what is the domain of discourse?
what are the set of objects in the world
- what does your vocabulary mean?
maps constants to domain elements,
function symbols to (mathematical) functions
predicate symbols to relations

Environment says:

- what is value of each variable?

The semantics of term:

$\llbracket t \rrbracket_{\mathcal{I}}$ given a term t , maps it to a domain element

$\llbracket \phi \rrbracket_{\mathcal{I}}$ given a formula ϕ , maps it to a truth value

For propositional logic, the semantics
is a computable function.

You can write a computer program that
implements prop. logic semantics,
and compute validity.

For predicate logic, this no longer holds.
 The semantics of predicate logic is a
 mathematical function, but not a
 computable function.
 We can't write a computer program
 to calculate the truth value of a
 pred logic formula.

Instead, the point of predicate logic
 is not to compute,
 but to give a precise description of
 what the logic *means*.
 This is useful is want to convince
 ourselves that we got it right.

This formula is satisfiable:

$\forall x \forall y. x=y$
 ..but not valid.

satisfiable
 true in a model with domain $\{x\}$ for some x
 not valid (false in any model with >1 element)

...we can instead say:

$(\forall x. x = c) \models \forall x \forall y. x=y$
 $\wedge$ here, instead of specifying a model directly,
 we've *axiomatized* the model as a theory.

Now, we essentially prove "in every theory that
 satisfies our axioms, the conclusion holds".

$T \models \forall x \forall y (y = x + 1 \rightarrow y \geq x)$
 ...where T is the set of all Peano axioms

We've seen natural deduction for prop logic.
 For each operator ($\neg, \wedge, \vee, \dots$) we have
 introduction and elimination rules that tell
 us how we can use proofs that we already
 have to get new proofs about that operator.

$\vdash A \quad \vdash B$	$\vdash A \wedge B$	$\vdash A \wedge B$
$\vdash A \wedge B$	$\vdash A$	$\vdash A$
$\wedge I$	$\wedge E$	$\wedge E$

Natural deduction for predicate logic
 is the same, but adds rules for
 $= \quad \forall \quad \exists$

Propositional ND is kind of a toy.

- We don't *really* need it,
 there's very mature algorithms
 for reasoning about propositional
 formulas, none of which use ND

For predicate logic,
 there are no algorithms.
 (If there were, mathematicians would be
 unemployed)

Transitivity:

prove $t = u, u = v \vdash t = v$

| 1. $t = u$
2. $u = v$
3. $t = v$ =E-2 2,1

$A \equiv \cdot = v$
 $A(u) \equiv u = v$
 $A(t) \equiv t = v$

$t = u \quad A(u)$

$$\frac{}{A(t)} =E-2$$

Symmetry:

prove $t = u \vdash u = t$ | 1. $t = u$

| -

| 2. $t = t$ $=I$ | 3. $u = t$ $=E-1 \quad 1,2$ $A \equiv \cdot = t$ $A(t) \equiv t = t$ $A(u) \equiv u = t$

$$\frac{t = u \quad A(t)}{A(u)} =E$$
| 1. $P(a)$

| -

| | $P(b)$

| | -

| | ... (* on this line, neither b nor a is arbitrary *)

| ... (* on this line, b is arbitrary *)

Let's "prove" by natural deduction
using a fake $\forall I$ rule that doesn't
require "c is free in A"

 $\vdash \forall x. x = c$

|

| -

| $c = c$ $=I$ | $\forall x. x = c$ fake $\forall I$ ^ here we used a context $A \equiv \cdot = c$

Why no function symbols?

 $x+y \geq z$

we could instead have a ternary predicate symbol

$P(x,y,z)$ where
the intended meaning of P is $x+y \geq z$

 $\exists y \forall x P(x,y) \models \forall x \exists y P(x,y)$

Quantifier order matters